Q1. Model Landscape for Local Deployment

The preferred model depends heavily on use case and hardware, as a personal machine and a company server call for different answers. Anyway, using different specialized models for different tasks is usually a good approach for both scenarios since it most likely increases the quality of the answer.
For an average computer science student, the limiting factor is probably the GPU. Assuming a setup with a small to medium GPU (~8-12gB VRAM), a reasonable configuration could include one small model for code completion and one larger model for code generation, reasoning, and debugging. Qwen2.5-Coder-1.5B-Instruct handles inline completion well, and Qwen2.5-Coder-7B-Instruct with Q4 quantization covers the heavier tasks. Whether a general chat model is worth running locally is debatable, since the performance on that personal machine will be limited due to limitations in parameter size. For coding, advantages of faster inference and keeping the code as well as internal structures locally motivate local deployment of a model, even though a model of that size might be limited especially in reasoning and also agent tasks. For general chat models, it can be argued that a model of this size without being able to search the internet is unlikely to give satisfying and correct answers. However, this argument shifts when considering the server setting, where larger models become viable. A 32B instruction-tuned Qwen2.5-Coder or the more recent Qwen3-Coder-Next are strong candidates for code while GPT-OSS 120B would be interesting for a general assistant. Additional advantages of local hosting include version-pinned reproducibility, controlled logging, no API costs, and the option to fine-tune for specific tasks. Finally, when deciding for a model to host, important factors beyond benchmarks can also be the general feedback from the community or literature coverage and information of the model. Beyond the obvious requirement of weights being open source, insight into the training process can help to determine the quality.

Q2. One GPU — Best Model

The H200's 141 GB HBM3e technically allows loading a 70.5B parameter model in FP16 or 141B in Q8 quantization, but this ignores the additional runtime memory required for activations and the KV cache, which can consume up to ~20% of VRAM. KV cache size scales linearly with transformer layers, hidden dimensions, KV-cache precision, and sequence length. And while optimized architectures can reduce this load via Grouped Query Attention (GQA) or Multi-head Latent Attention (MLA), the parameters to be used still should not fill up the entire VRAM. Additionally, in line of choosing the best-quality model, a reasonable consideration is opting for Q8 quantization which will enable using a larger model in general while dropping comparably little of the performance.
A clean practical solution could be Qwen3-80B (A3B MoE variant) with Q8 quantization. Weights occupy roughly 80–85 GB, leaving ~50 GB for the KV cache. This is sufficient for multiple parallel conversations and the long contexts the model natively supports. Additionally, the MoE architecture is particularly attractive here because it reduces the number of active parameters per forward pass drastically, reducing compute and memory pressure while the full parameter space remains available for routing. To make fuller use of the available VRAM, running GPT-OSS 120B with Q8 would fit as well, also allowing for larger context size. But having multiple conversations simultaneously is where the limits of the VRAM would be reached.

Q3. Speed and Optimization

For Qwen3-80B A3B at Q8 on a single H200, a realistic baseline is around 60 tokens/sec for a single user at 4096 context length according to online benchmarks. The primary bottleneck is memory bandwidth rather than compute: at each decode step, all model weights must be loaded from VRAM to process a single token. The H200's 4.8 TB/s helps significantly here. Time to first token (TTFT) is dominated by the prefill phase, which processes the full input sequence at once and scales with

context length. Continuous batching, as implemented in vLLM, inserts new requests into the batch as slots free, yielding roughly linear throughput scaling with batch size at the cost of slightly increased individual latency. PagedAttention avoids memory fragmentation by allocating KV cache in fixed pages, allowing more sequences to share GPU memory efficiently. That becomes important for parallel requests. That could also happen if a single user uses multiple conversations at once. Speculative decoding uses a small draft model to propose tokens ahead, the large model verifies them in parallel. This can lead to two to three times speedup on predictable outputs like code, with negligible quality loss. Finally and most importantly, the MoE architecture already provides a big structural advantage. Instead of the full 80B, only 3B active parameters are passed per decode step which leads to a substantially smaller computation load per step than a comparably-sized dense model and therefore a higher throughput.

Q4. Full Inference Stack

A well-working stack includes vLLM as the inference engine, fronted by LiteLLM as a model router, behind nginx for TLS and rate limiting and with Open WebUI as the interaction layer. Further, PostgreSQL can handle append-only audit logging per request, and Prometheus with Grafana can be added to cover operational metrics. Everything can be in Docker Compose where explicit GPU allocations can be set via the NVIDIA Container Toolkit. This is very convenient because the full environment can be reconstructed and run from a single compose file. The single components are chosen for the following reasons:
vLLM is the inference engine of choice because it features PagedAttention and continuous batching, which sustain throughput under concurrent load. LiteLLM exposes a single OpenAI-compatible endpoint regardless of how many models run behind it and handles per-user rate limiting which becomes very useful when maintaining different specialized models. The logging is split into two parts, Prometheus is used for logging of operational metrics such as GPU utilization, KV cache fill and latency. PostgreSQL handles the more persistent and auditable records, such as a log of prompt, usage and model response. Open WebUI can cover multimodal inputs and LDAP/SSO out of the box so there is no further need to build a custom UI just yet.

Q5. Dream Hardware — DGX B300

The DGX B300's 2.1 TB total GPU memory and 14.4 TB/s NVLink bandwidth opens model classes that simply cannot run at quality on smaller nodes. For a possible starting architecture one could allocate five GPUs to a full-precision multimodal flagship model, for example Qwen3.5-397B or comparable. This model can provide strong reasoning and vision-language understanding which is directly relevant for radiology imaging and related research. Two remaining GPUs could host smaller specialized models, such as Meditron-70B for medical domain tasks and Qwen3-Coder-Next for coding, depending on the departments demand. The final GPU is reserved for research workloads like fine-tuning, kept isolated to avoid introducing latency for production users.
To provide a convenient setup serving the whole department, LiteLLM is a valuable framework. It provides a single unified endpoint with per-user and per-model rate limiting, preventing large requests from starving the queue.

As a final note: The LLM landscape moves quickly, and which makes keeping up with all new developments challenging but interesting. Useful sources include benchmark leader boards and community forums such as Hugging Face's model hub as well as technology news. To get an overview, it can be very helpful to consult the LLM platform of choice (while keeping in mind not to trust any information blindly).